

Filters in Spring Boot — Detailed Notes

1. What is a Filter?

Definition: A **Filter** is a Java object (part of the **Servlet API**, not Spring-specific) that performs filtering tasks on either the **request** to a resource, the **response** from a resource, or both — **before** the request reaches a Servlet (like `DispatcherServlet`) and **after** the response leaves it.

- Package: `jakarta.servlet.Filter` (formerly `javax.servlet.Filter`)
- Filters operate at the **Servlet Container level** (Tomcat), which means they run **before** Spring's `DispatcherServlet` even gets involved.
- Multiple filters can be chained together — this is called a **Filter Chain**.

Key Idea: Filters are a **pre-processing and post-processing mechanism** that sits between the client and the servlet, intercepting every request/response that matches its configured URL pattern.



2. The `Filter` Interface

Every filter must implement `jakarta.servlet.Filter`, which has 3 methods:

```
public interface Filter {
    default void init(FilterConfig filterConfig) throws ServletException {}

    void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
        throws IOException, ServletException;

    default void destroy() {}
}
```

Method	Purpose
<code>init()</code>	Called once , when the filter is first initialized by the container (before it starts processing requests). Used for setup/config.
<code>doFilter()</code>	Called for every matching request . Contains the actual filtering logic — this is the core method.
<code>destroy()</code>	Called once , when the filter is being removed from service (app shutdown). Used for cleanup.

Note: This lifecycle is very similar to the Servlet lifecycle (`init` → `service` → `destroy`) — because filters are managed by the same Servlet Container.

3. Creating a Custom Filter — Example

```

@Component
public class LoggingFilter implements Filter {

    @Override
    public void init(FilterConfig filterConfig) throws ServletException {
        System.out.println("LoggingFilter initialized");
    }

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
        throws IOException, ServletException {

        HttpServletRequest httpRequest = (HttpServletRequest) request;

        System.out.println("Incoming Request: " + httpRequest.getMethod() + " " + httpRequest.getRequestURI());

        long startTime = System.currentTimeMillis();

        // Pass control to the next filter in the chain (or the servlet, if this is the last filter)
        chain.doFilter(request, response);

        long endTime = System.currentTimeMillis();
        System.out.println("Request processed in " + (endTime - startTime) + " ms");
    }

    @Override
    public void destroy() {
        System.out.println("LoggingFilter destroyed");
    }
}

```

Key Line: `chain.doFilter(request, response);`

- This line is **critical** — it passes control to the **next filter** in the chain (or to the servlet if this is the last one).
- Code **before** this line runs during the **request phase** (pre-processing).
- Code **after** this line runs during the **response phase** (post-processing) — after the Controller has finished and the response is on its way back.
- **If you forget to call `chain.doFilter()`**, the request is effectively **blocked** — it never reaches the Controller, and the client gets no meaningful response (this is actually how filters are used to reject unauthorized requests).

4. What is `FilterChain`?

Definition: `FilterChain` is an object provided by the Servlet Container representing the **sequence of filters** configured for a particular request, plus the final target servlet.

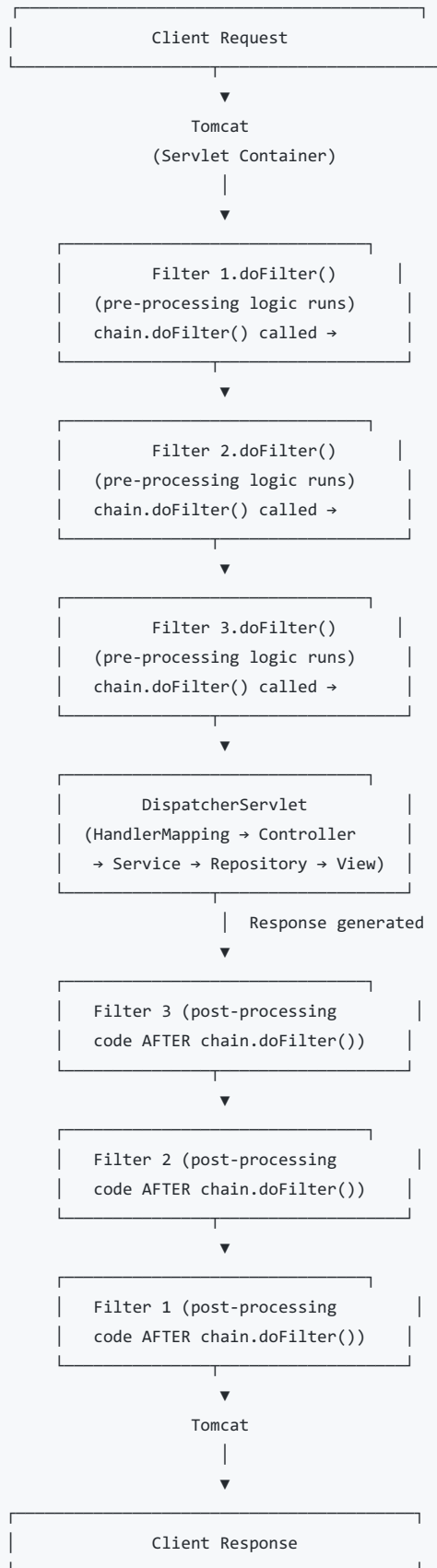
```

public interface FilterChain {
    void doFilter(ServletRequest request, ServletResponse response)
        throws IOException, ServletException;
}

```

- Each filter receives a reference to this chain.
- Calling `chain.doFilter(request, response)` tells the container: *"I'm done with my part — pass control to the next filter (or the servlet, if I'm the last one)."*
- This is what makes Filters composable — you can have any number of filters, each doing one specific job, chained together in sequence.

5. Full Filter Chain Flow — Detailed



Important Detail – Order of Execution:

- Filters run in the order they're registered for the **request** phase (Filter1 → Filter2 → Filter3).
- But for the **response** phase, execution happens in **reverse order** (Filter3 → Filter2 → Filter1) – this is because each filter is "wrapped around" the next, like layers of an onion.

Request: F1 → F2 → F3 → Servlet

Response: F1 ← F2 ← F3 ← Servlet

This is often called the **"Chain of Responsibility"** design pattern.

6. ServletRequest and ServletResponse in Filters

- The `doFilter()` method receives generic `ServletRequest` and `ServletResponse` objects (not `HttpServletRequest` / `HttpServletResponse` directly) – because the Servlet API is protocol-agnostic in theory.
- In practice, for web apps, you almost always cast them:

```
HttpServletRequest httpRequest = (HttpServletRequest) request;
HttpServletResponse httpResponse = (HttpServletResponse) response;
```

- A Filter can:
 - **Read** request data (headers, params, body) – for logging, auth checks, etc.
 - **Modify** the request – by wrapping it in a custom `HttpServletRequestWrapper` (e.g., to modify headers before it reaches the Controller)
 - **Modify** the response – by wrapping it in a `HttpServletResponseWrapper` (e.g., to add custom headers, compress content)
 - **Block** the request entirely – by not calling `chain.doFilter()` and directly writing an error response

Example — Blocking unauthorized requests:

```
@Override
public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
    throws IOException, ServletException {

    HttpServletRequest httpRequest = (HttpServletRequest) request;
    HttpServletResponse httpResponse = (HttpServletResponse) response;

    String token = httpRequest.getHeader("Authorization");

    if (token == null || !isValidToken(token)) {
        httpResponse.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
        httpResponse.getWriter().write("Unauthorized");
        return; // chain.doFilter() is NOT called – request stops here
    }

    chain.doFilter(request, response); // valid token, proceed
}
```

7. How to Register a Filter in Spring Boot

Method A: `@Component` + `@Order` (simplest)

```
@Component
@Order(1)
public class LoggingFilter implements Filter { ... }
```

- Spring Boot auto-detects any `Filter` bean and registers it automatically.
- `@Order` controls the sequence when multiple filters exist (lower number = higher priority = runs first).

Method B: `FilterRegistrationBean` (more control)

```
@Configuration
public class FilterConfig {

    @Bean
    public FilterRegistrationBean<LoggingFilter> loggingFilter() {
        FilterRegistrationBean<LoggingFilter> registrationBean = new FilterRegistrationBean<>();
        registrationBean.setFilter(new LoggingFilter());
        registrationBean.addUrlPatterns("/api/*"); // apply only to specific URLs
        registrationBean.setOrder(1); // execution order
        return registrationBean;
    }
}
```

- Gives fine-grained control: specific URL patterns, order, init parameters — useful when you don't want the filter applied globally to `/*`.

Method C: `@WebFilter` (Servlet-style, less common in Spring Boot)

```
@WebFilter(urlPatterns = "/api/*")
public class LoggingFilter implements Filter { ... }
```

- Requires `@ServletComponentScan` on your main application class to be detected.
- Works via the same annotation-scanning mechanism discussed earlier (Servlet 3.0 Pluggability API).

8. Common Real-World Use Cases for Filters

Use Case	Description
Authentication/Authorization	Check JWT tokens, API keys, session validity before allowing request to proceed
Logging	Log every incoming request/outgoing response (method, URI, status, response time)
CORS Handling	Add <code>Access-Control-Allow-Origin</code> and related headers
Request/Response Modification	Wrap and modify headers, compress/decompress content
Input Sanitization	Clean/validate raw request data before it reaches the app (e.g., XSS prevention)
Rate Limiting	Track and limit number of requests from a client within a time window
Character Encoding	Set encoding (<code>CharacterEncodingFilter</code> — a built-in Spring filter)
Caching	Serve cached responses without hitting the Controller
Correlation ID / Tracing	Attach a unique request ID for distributed tracing/logging across microservices

9. Filters vs Interceptors vs `@ExceptionHandler` (Big Picture)

This is a very commonly confused topic — here's the full comparison:

Aspect	Filter	Interceptor (<code>HandlerInterceptor</code>)	<code>@ExceptionHandler</code>
Belongs to	Servlet API (Java EE)	Spring MVC	Spring MVC
Runs relative to <code>DispatcherServlet</code>	Before <code>DispatcherServlet</code> (outside Spring context)	After <code>DispatcherServlet</code> , around the Controller call	Only on exceptions, handled by <code>DispatcherServlet</code>
Has access to Spring beans/context	Limited (needs to be a <code>@Component</code> to access beans)	Full Spring context access	Full Spring context access
Granularity	URL pattern-based (<code>/api/*</code>)	Can be Controller/handler-method specific	Exception-type specific
Key methods	<code>init()</code> , <code>doFilter()</code> , <code>destroy()</code>	<code>preHandle()</code> , <code>postHandle()</code> , <code>afterCompletion()</code>	Any custom method annotated <code>@ExceptionHandler</code>
Typical use	Auth, logging, CORS, encoding (broad, low-level concerns)	Logging with Spring context, request modification tied to specific handler logic, auth tied to annotations	Centralized error response formatting

Combined Request Flow (Full Picture):

```

Client
↓
Filter 1 → Filter 2 → Filter 3   (Servlet-level, outside Spring MVC)
↓
DispatcherServlet
↓
Interceptor.preHandle()          (Spring MVC-level, before Controller)
↓
Controller method executes
↓
Interceptor.postHandle()         (after Controller, before view rendering)
↓
View rendered / Response built
↓
Interceptor.afterCompletion()     (after complete response is ready)
↓
DispatcherServlet
↓
Filter 3 ← Filter 2 ← Filter 1   (post-processing, reverse order)
↓
Client

```

(If an exception occurs anywhere in the Controller/Service/Repository chain, it's intercepted by `HandlerExceptionResolver` and routed to a matching `@ExceptionHandler` before the response is finalized — this happens between the Controller execution and Interceptor's `postHandle()` / `afterCompletion()`.)

10. `OncePerRequestFilter` (Spring's Enhanced Filter Base Class)

Spring provides an abstract class `OncePerRequestFilter` (in `org.springframework.web.filter` package) that's commonly used instead of implementing `Filter` directly.

Why use it?

- Guarantees the filter's logic executes **only once per request** — even in cases involving internal request dispatching/forwarding (where a raw `Filter` might accidentally run twice).
- Provides typed `HttpServletRequest` / `HttpServletResponse` directly — no manual casting needed.

```

@Component
public class LoggingFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException {

        System.out.println("Request URI: " + request.getRequestURI());

        filterChain.doFilter(request, response);

        System.out.println("Response Status: " + response.getStatus());
    }
}

```

This is the **recommended approach** for writing filters in modern Spring Boot applications (especially common in Spring Security's `JwtAuthenticationFilter`, etc.)

11. Built-in Filters Provided by Spring

Spring/Spring Boot ships with several ready-made filters you can enable via configuration:

Filter	Purpose
<code>CharacterEncodingFilter</code>	Sets character encoding (e.g., UTF-8) for requests/responses
<code>CorsFilter</code>	Handles Cross-Origin Resource Sharing headers

HiddenHttpMethodFilter Filter	Allows HTML forms to simulate PUT/DELETE via a hidden <code>_method</code> param
RequestContextFilter	Exposes request attributes to the current thread (useful for non-web-aware beans)
Spring Security's FilterChainProxy	An entire chain of security-related filters (authentication, CSRF, session management, etc.) – this is itself registered as a single Filter with Tomcat, but internally runs its own mini filter chain

12. Quick Summary (for revision)

- A **Filter** is a Servlet API component that intercepts requests/responses **before and after** they reach `DispatcherServlet` – operating at the Servlet Container level.
- Core method: `doFilter(request, response, chain)` – code before `chain.doFilter()` = pre-processing; code after = post-processing.
- **FilterChain** links multiple filters together; calling `chain.doFilter()` passes control to the next filter (or the servlet).
- **Request phase**: filters run in registered order. **Response phase**: filters run in **reverse** order (onion-layer pattern).
- Filters can **read, modify, or block** requests/responses entirely.
- **Registration methods**: `@Component + @Order`, `FilterRegistrationBean` (fine-grained control), or `@WebFilter` (needs `@ServletComponentScan`).
- **OncePerRequestFilter** is Spring's recommended base class – guarantees single execution per request and gives typed HTTP objects directly.
- **Common uses**: authentication, logging, CORS, encoding, rate limiting, request/response modification.
- **Filters vs Interceptors**: Filters run outside Spring MVC (broader, Servlet-level); Interceptors run inside Spring MVC (finer-grained, full Spring context access, tied to handler methods).
- **Full flow**: Client → Filters (forward order) → `DispatcherServlet` → `Interceptors` → `Controller` → `Interceptors` → `DispatcherServlet` → Filters (reverse order) → Client